Babbage's Hall

A Victorian hall of machines that compute · 20 October 2026

A Victorian engineering hall of machines that compute: cast-iron columns painted Brunswick green, with gilded capitals, gas lamps, a glass barrel vault, red brick, brass rails and encaustic tiles. **Every machine in it can be 3D printed and built**: each exhibit's folder holds its OpenSCAD sources, a script that makes its STL files, and a README saying what else to buy and how to put it together. What is printed is what is simulated.

The machines get **more interesting as you walk on**, from the Differential at the entrance to Babbage's own Difference Engine at the far end: a bay on the west side, then the one facing it on the east, and on. The bays still waiting for their machines stand covered, "in preparation". When an exhibit's script turns up in its folder, its bay comes to life.

Bay	Exhibit	
west 1	**The Differential**: bevel gears that add and subtract	`exhibits/Differential/`
east 1	Lehmer's number sieve	in preparation
west 2	**The Pascaline**: Pascal's adding machine, with his falling-weight carry	`exhibits/Pascaline/`
east 2	**The Domino Half Adder**: dominoes that add two bits	`exhibits/DominoAdder/`
west 3	The Digi-Comp II	in preparation
east 3	Kelvin's tide predictor	in preparation
west 4	The Differential Analyser	in preparation
east 4	The Difference Engine	in preparation

`MACHINES.md` lists every machine considered for the hall (47 of them), ranked by interest, with how each would be simulated and printed.

Running it

Open `hall.lua` in bpp, or from the repository's top folder run `release/bpp -f demo/WyomingWill/BabbagesHall/hall.lua`. Each exhibit also runs on its own from its folder.

Key	
`Tab`	walk on: the view from the entrance, the exhibits, the view from the title wall
`Shift+Tab`	walk back

Click the 3D view first so it has the keyboard. Everything else goes to the exhibit you're at, with the same keys as when it runs on its own; the Shortcuts pane lists them. `HALL_START=differential` (any part of a name) starts at that exhibit, and `HALL_START=end` at the title wall.

On Tab and Shift+Tab the camera flies to the next stop, out of the bay, along the aisle and into the next bay, in 1.2 to 3.5 seconds. The exhibit you're going to starts at once; its own camera moves wait until you've arrived. `HALL_FLY=0` jumps instead, as before.

How it works

The machinery is Euler's Gallery's, unchanged; the building around it is new.

- **Separate worlds.** Each exhibit runs in a Bullet world of its own, at its own coordinates, drawn in its bay. Only the exhibit you are standing at is stepped.
- **Sandboxes.** Each exhibit is loaded with its own globals, its own random numbers, and its own sliders, keys and help.
- **Files.** An exhibit finds its files from its own folder. `hallPath(name)` gives a file's path from bpp's folder; the exhibits use it to hand OpenSCAD the full path to their parts files.
- **Requirement:** a bpp with separate worlds (`v:newWorld`).
- **POV-Ray.** The hall renders in POV-Ray (File, or `bpp -e`) as on the screen, with a bpp that exports each object moved to where its world is drawn (`v:placeWorld`); an older one exports every exhibit at its own coordinates, piled up in the middle of the nave. The exhibits' ground planes, invisible on the screen, are left out: in POV-Ray a plane has no edges and would cover the tiled floor.

What it costs the computer

Only the exhibit you're standing at is stepped; the others wait where they are, and at the two views nothing is. And each exhibit **rests** when it has nothing to do: once its hand is idle and nothing has moved for half a second, it stops stepping until there's something to do again (a key, or the tour's next step). The physics is the expensive part. The exhibits take 2400 steps a simulated second, which the gear teeth and the dominoes need, and a machine standing still costs as much to step as one running. Resting changes nothing it computes: woken, it carries on from exactly where it was. Measured over a 70 s run on its own, the Differential took 79 s of computing without resting and 16 s with it (its tour ends after about 20 s); over 50 s, the Domino Half Adder took 165 s and 88 s.

The **cost meter** (after the Rec Room's) shows in the console every 10 seconds what each frame spends its time on, in milliseconds. Here, at the Differential during its tour, and then resting after it, on a computer drawing in software (no graphics card), so its drawing is slow:

```
HALL METER -- ms per frame, averaged over the last 10 s (HALL_METER=0: off)
  7 frames a second; at the Differential: resting (not stepping) in 0% of the frames
  physics 22.27   scripts 0.82   garbage (Lua's own)   drawing 25.28
  busy    48.4 ms of each 153.5 ms frame (32%); longest frame 219 ms
  outside the hall's code 105.2 ms a frame (bpp itself, the graphics driver, waiting for the screen)

HALL METER -- ms per frame, averaged over the last 10 s (HALL_METER=0: off)
  9 frames a second; at the Differential: resting (not stepping) in 100% of the frames
  physics 0.01   scripts 0.10   garbage (Lua's own)   drawing 22.66
  busy    23.9 ms of each 113.1 ms frame (21%); longest frame 148 ms
  outside the hall's code 89.2 ms a frame (bpp itself, the graphics driver, waiting for the screen)
```

- **physics:** the exhibit's steps, and bpp's own.
- **scripts:** the exhibit's own Lua, every frame.
- **garbage:** Lua's own collector by default, which works a little at a time. `HALL_GC_MS=2` collects up to 2 ms every frame instead, as the Rec Room does, so garbage never piles up into one long collection; here it cost 0.4 ms every frame, so it's off unless the freeze log shows pauses.

- **drawing:** the processor's side of drawing the frame. The graphics card's own work isn't in it; bpp's `v.drawTiming = true` shows that.
- **busy:** all of it, against the time a frame has. bpp runs on one processor core, so a busy core shows as 100% in `top`.
- **outside the hall's code:** the rest of the frame: bpp's own work, the graphics driver finishing the picture, and waiting for the screen.

`HALL_METER_S` sets how often it prints; `HALL_METER_FILE=path` also appends each report to that file.

A frame that takes longer than a quarter of a second is noted in the console with the time of day (`HALL_FREEZE_MS`, 250; 0: never). `HALL_METER_S` sets how often the meter prints.

The placards' lettering is made by `python3 make-labels.py` (needs Pillow, NumPy and the DejaVu fonts) into `hall-meshes/`.

Every `.md` file has a typeset `.pdf` beside it; `./make-pdfs.sh` rebuilds them all (needs `pip install markdown playwright`). The STL files are not kept in the repository: each exhibit's `make-stls.sh` makes them from its OpenSCAD sources.

Notes for exhibit authors

Two things about bpp that cost time, and how the exhibits handle them:

- **OpenSCAD caching.** bpp caches each OpenSCAD part by the text asked for, not by what the included files say, so an edited `.scad` file is not seen. The exhibits put a checksum of their parts files into that text.
- **Timing of settings.** An OpenSCAD mesh's place has to be set after it is added to the world. Its `collides` has to be set before: collision groups are fixed when a body is added.
- **Gear contact.** Bevel teeth as meshes (GImpact) jammed and slipped. As compounds of convex hulls (one per tooth, plus the body) they mesh well. See `exhibits/Differential/FINDINGS.md`; `parts/make-hulls.py` there makes them from the same OpenSCAD source.
- **`.scad` files and git.** bpp's top-level `.gitignore` ignores `*.scad`, so a new exhibit's parts files are silently left out of a commit. The hall's own `.gitignore` has `!*.scad` to keep them; check with `git status` that they are there.
- **One source of geometry.** The Pascaline keeps every moving part's shape as primitives in a Lua file; the simulation builds its bodies from them and writes `geometry.scad`, from which OpenSCAD builds the printed parts.
- **Centre of mass.** A Bullet body's origin is where gravity acts, so a lever built from an off-centre mesh must have its origin moved to its centre of mass (the Pascaline's `bodyOf` does this).
- **Hinge angles.** `getHingeAngle` is clockwise-positive about the hinge's axis, the opposite of a right-handed turn.